

Pincher Complete Guide

React frontend, HTML/CSS/JavaScript structure, FastAPI backend, weather-aware wardrobe logic, and ML training plan

A teammate-friendly handoff that explains the app from the first setup step to the complete version.

Audience	A teammate who has no prior context and needs a clear, practical walkthrough.
Product	Pincher, a wardrobe assistant for outfit selection and style guidance.
Frontend	Next.js + React for dynamic screens, with HTML structure, CSS styling, and JavaScript behavior.
Backend	FastAPI with PostgreSQL, JWT auth, and outfit-scoring endpoints.
Core rule	Final outfit output must come from the wardrobe only. Do not use generated image output.
Date	August 22, 2026

Scope note: the current repository already contains the new Next.js app and a working FastAPI backend, but it also still includes legacy backend TypeScript files. This guide explains the active path and points out what should be retired in the complete version.

1. How To Use This Guide

Read the problem statement first, then the repository map, and then the build steps. Each step explains what the problem is, how the solution works, which files matter, and what success looks like. If a section mentions a future folder that is not yet in the repo, that means the guide is describing the recommended complete version rather than the current partial implementation.

Use the guide as both a learning document and a build plan. A new teammate should be able to understand the product goal, the stack, the data model, the weather logic, and the ML plan without needing to ask for a separate architecture walkthrough.

2. Product Overview

Pincher is a wardrobe intelligence app. A user stores clothing items, answers a few outfit questions, and receives outfit recommendations based on the items they already own. The final product is not an image generator. It is a decision engine that selects real wardrobe pieces, explains why they work together, and adapts the result to the weather, the occasion, and the user's color preferences.

This is why the frontend focuses on wardrobe browsing, outfit building, and result explanation, while the backend focuses on authentication, item storage, ranking, and weather-aware scoring. React is used where state and routing help, but the actual screens still follow plain HTML structure, CSS styling, and JavaScript behavior inside the JSX components.

3. Problem Statement

People often own many clothes but still struggle to answer simple questions such as: What should I wear today? Which pieces match the weather? Which colors look balanced together? Which outfits fit a formal or casual situation? Without a system, the closet becomes a collection of disconnected items instead of a usable wardrobe.

A second problem is that many fashion tools try to solve the issue by generating fake outfit images. That looks nice, but it is not useful when the user wants to wear clothes they already own. Pincher should therefore choose from the actual wardrobe, not invent a look that cannot be worn.

4. Target Solution

The complete solution is a React frontend backed by FastAPI and PostgreSQL. The frontend collects the user's choices, shows wardrobe items, and presents ranked outfit options. The backend stores users, profiles, wardrobe items, outfit collections, weather context, and recommendation results. A scoring

engine combines rules, color harmony, weather rules, and ML ranking so the app can give useful results even before the model becomes sophisticated.

The most important design rule is that every output must stay grounded in the wardrobe database. If a user has no blue shirt, the app should not show a blue shirt just because the recommendation model thinks it would look nice. The result must come from the user's actual items, with color and weather logic only used to rank and explain the choices.

5. Current Repository Snapshot

The repository has two active roots: ``frontend/`` and ``backend/``. The old static HTML/CSS/JS frontend has been deleted, so the Next.js app under ``frontend/app/`` is now the source of truth. The backend is mostly FastAPI, but there are still legacy Express/TypeScript files under ``backend/src/`` and ``backend/dist/`` that should be treated as archive material in the final version.

Active frontend	frontend/ - Next.js app router, React pages, styling, and shared browser logic.
Active backend	backend/main.py plus db.py, auth.py, init_db.py, and schema.sql.
Legacy backend	backend/src/ and backend/dist/ - old Express/TypeScript implementation still present in the repo.
Generated folders	node_modules/, __pycache__/, .next/, and dist/ should not be treated as source code.

6. Frontend Folder And File Map

The frontend is a Next.js app using React 19, Framer Motion, lucide-react icons, and Tailwind CSS 4. The main job of the frontend is to collect user input, let the user inspect wardrobe data, and show ranked outfit results clearly. The folder map below explains the active files by purpose, not just by name.

6.1 App Shell Files

Path	Purpose
frontend/app/layout.jsx	Root app wrapper. Loads Google fonts, sets metadata, and provides the base HTML structure.
frontend/app/globals.css	Global design tokens, Tailwind theme values, color palette, and a few utility animations.
frontend/app/page.jsx	Landing page. Shows the marketing home screen and animated product intro.
frontend/app/loading.jsx	Route loading state used while pages are resolving.

frontend/app/not-found.jsx	Fallback page for unknown routes.
frontend/app/favicon.ico	Browser tab icon for the app.

6.2 Authentication Pages

Path	Purpose
frontend/app/login/page.jsx	Login form and authentication entry point.
frontend/app/signup/page.jsx	Account creation flow.
frontend/app/forgot-password/page.jsx	Request reset instructions or a reset code.
frontend/app/reset-password/page.jsx	Set a new password after recovery.
frontend/app/otp/page.jsx	One-time code verification if the flow needs it.

6.3 Dashboard Pages

Path	Purpose
frontend/app/dashboard/layout.jsx	Shared dashboard shell with sidebar, header, profile area, and mobile menu.
frontend/app/dashboard/page.jsx	Dashboard home or overview screen.
frontend/app/dashboard/wardrobe/page.jsx	Wardrobe catalog, filters, categories, and item browsing.
frontend/app/dashboard/outfit-builder/page.jsx	Interactive outfit assembly and question flow.
frontend/app/dashboard/upload/page.jsx	Upload screen for new wardrobe items.
frontend/app/dashboard/results/page.jsx	Final outfit result and explanation screen.
frontend/app/dashboard/history/page.jsx	Past outfit history and previous runs.
frontend/app/dashboard/saved/page.jsx	Saved outfits and favorites.
frontend/app/dashboard/recommendations/page.jsx	Ranked suggestions based on wardrobe and weather.

frontend/app/dashboard/insights/page.jsx	Style insights and behavior summaries.
frontend/app/dashboard/analytics/page.jsx	Scoring and usage metrics.
frontend/app/dashboard/profile/page.jsx	User profile, persona, and style preferences.
frontend/app/dashboard/settings/page.jsx	App preferences, privacy, and account settings.

6.4 Shared Frontend Helpers

Path	Purpose
frontend/app/lib/imageUtils.js	Image compression, thumbnail creation, and dominant-color extraction for uploaded wardrobe images.
frontend/app/lib/colorHarmony.js	Color conversions, harmony scoring, compatibility scoring, and color naming.
frontend/app/lib/deepfashion2.js	Clothing taxonomy, style attributes, occasion labels, personas, and color-season data.

Important: `deepfashion2.js` currently contains a prompt generator for image generation. In the complete version, keep the taxonomy and color logic but remove or disable the prompt generator because the final product should not generate outfit images.

6.5 Frontend Tooling And Documentation

Path	Purpose
frontend/package.json	Frontend scripts and dependencies for Next.js, React, Tailwind, Framer Motion, and icon support.
frontend/package-lock.json	Lock file for the frontend dependency tree.
frontend/next.config.mjs	Next.js configuration.
frontend/postcss.config.mjs	PostCSS config for Tailwind processing.
frontend/eslint.config.mjs	Linting rules for the app.
frontend/README.md	Starter Next.js readme. It should eventually be rewritten for the actual Pincher app.
frontend/CLAUDE.md	Assistant-facing instructions for the frontend workspace.
frontend/AGENTS.md	Additional assistant or workflow notes for collaborators.

frontend/public/	Static assets such as SVG icons used by the app.
------------------	--

7. Backend Folder And File Map

The backend is a FastAPI application that talks to PostgreSQL, hashes passwords, issues JWT tokens, and stores wardrobe and outfit data. The code currently lives mostly in a few small Python files, which makes it easy to understand but also means the complete version should eventually be split into routers, services, and schemas once the app grows.

7.1 FastAPI Runtime Files

Path	Purpose
backend/main.py	Main FastAPI app. Defines health, auth, wardrobe, and outfit endpoints plus CORS and exception handling.
backend/db.py	Connection pool helper for PostgreSQL and a cursor context manager.
backend/auth.py	JWT creation and bearer-token validation for protected routes.
backend/init_db.py	Database bootstrap script that loads `schema.sql`.
backend/requirements.txt	Python dependencies for FastAPI, Uvicorn, PostgreSQL, JWT, dotenv, and bcrypt.
backend/README.md	Quick start notes for the backend.
backend/package.json	Wrapper script file that launches Uvicorn. It is not the application runtime itself.
backend/package-lock.json	Lock file for the wrapper package. It is not important to the FastAPI logic.

7.2 Schema And Data Files

Path	Purpose
backend/src/schema.sql	PostgreSQL schema for users, profiles, wardrobe items, and outfit collections.
backend/src/	Legacy TypeScript Express-era backend source. Treat this as archive material in the final FastAPI version.
backend/dist/	Compiled output from the old TypeScript backend. It should not be treated as source code.

The schema still includes `generated\_image\_url` and `generation\_prompt` in `outfit\_collections`. Those fields made sense when the product used image generation, but the complete wardrobe-only version should remove them or rename them so the data model no longer implies fake rendered outfits.

7.3 Legacy Backend Files

Path	Purpose
backend/src/index.ts	Old Express server entry point.
backend/src/db.ts	Legacy TypeScript database helper.
backend/src/init-db.ts	Legacy TypeScript database bootstrap script.
backend/src/middleware/auth.ts	Legacy authentication middleware.
backend/src/routes/auth.ts	Legacy Express auth routes.
backend/src/routes/wardrobe.ts	Legacy wardrobe routes.
backend/src/routes/outfits.ts	Legacy outfit routes.

These files are still useful as a reference for route design, but they are not the active runtime path once the FastAPI backend is the source of truth. In the complete version, keep one backend architecture and retire the duplicate one so the team does not waste time debugging two implementations at once.

7.4 Recommended Future Backend Structure

Recommended folder	Why add it
backend/app/routers/	Split auth, wardrobe, outfits, and weather routes into separate files.
backend/app/services/	Keep weather lookup, ranking, and ML inference logic out of the route handlers.
backend/app/schemas/	Store Pydantic request and response models here.
backend/app/core/	Settings, security, constants, and dependency helpers.
backend/ml/	Model training scripts, preprocessing, evaluation, and exported artifacts.
backend/tests/	API tests, unit tests, and regression checks.

8. Local Setup And Run Order

A new teammate should be able to start the project in a predictable order: install dependencies, configure environment variables, initialize the database, then start backend and frontend servers. If this order is unclear, the entire project feels harder than it really is.

Step 1 - Install the frontend and backend dependencies

Problem: The app depends on two runtimes: a Node-based frontend and a Python backend. If dependencies are missing, nothing else can be tested.

Solution: Run the frontend install command in ``frontend/`` and the Python install command in ``backend/``. Keep the installs separate so the Node package tree does not pollute the Python environment.

Files: ``frontend/package.json``, ``frontend/package-lock.json``, ``backend/requirements.txt``, and ``backend/package.json`` as the backend wrapper script.

Done when: Both environments can start without missing-package errors.

Step 2 - Configure environment variables

Problem: The backend cannot connect to PostgreSQL or sign JWT tokens without secrets.

Solution: Create a ``.env`` file for the backend with ``DATABASE_URL`` and ``JWT_SECRET``. If the weather service is added later, store that API key in the same pattern. The frontend should also know the API base URL if it is not using the same origin.

Files: Backend ``.env``, backend ``auth.py``, backend ``db.py``, and the frontend API helpers you may add later.

Done when: The backend starts and can read the database connection string and JWT secret.

Step 3 - Initialize the database

Problem: The app needs tables before any signup or wardrobe flow can work.

Solution: Run the schema bootstrap script so PostgreSQL gets the ``users``, ``user_profiles``, ``wardrobe_items``, and ``outfit_collections`` tables.

Files: Backend ``init_db.py`` and ``src/schema.sql``.

Done when: The schema exists and the database accepts reads and writes.

Step 4 - Start the backend API

Problem: The frontend cannot authenticate or fetch wardrobe data until the API is running.

Solution: Start Uvicorn from the backend wrapper scripts. Keep the API port stable so the frontend can call it consistently during local development.

Files: Backend ``main.py``, ``db.py``, ``auth.py``, and ``package.json``.

Done when: The ``/health`` endpoint returns a healthy response.

Step 5 - Start the frontend

Problem: Users need a browser UI to interact with the system.

Solution: Run the Next.js dev server in `frontend/` and open the app in the browser. The root landing page and dashboard pages should render cleanly before any API integration work is considered complete.

Files: Frontend `app/layout.jsx`, `app/page.jsx`, `app/dashboard/layout.jsx`, and the dashboard route files.

Done when: The home page and dashboard shell load without layout bugs.

9. Weather-Aware Outfit Logic

This is one of the most important features in the complete version. The app should not only know what items exist in the wardrobe; it should also know what the weather feels like and which items are appropriate for that context. The result is a recommendation that feels practical rather than cosmetic.

The weather-aware flow should ask the user for a location or use browser location permission, then fetch the current weather. From that weather response, extract temperature, feels-like temperature, humidity, wind, rain probability, and the condition code. Use those values to decide whether the outfit should be light, layered, water-resistant, breathable, or formal.

Weather condition	Wardrobe response	Why it matters
Hot and sunny	Prefer light fabrics, short sleeves, breathable materials, and lighter colors.	Comfort and heat management matter more than layering.
Cold or windy	Prefer layers, outerwear, heavier fabrics, boots, and longer coverage.	Warmth and wind protection matter.
Rainy or humid	Prefer closed shoes, water-resistant outerwear, and quick-dry materials.	Some fabrics and shoes perform badly in wet conditions.
Very formal event	Prefer structured items, clean lines, and restrained palettes even if weather is mild.	Dress code still matters alongside weather.
High humidity	Prefer breathable fabrics and relaxed fits.	The outfit should stay comfortable for longer wear.

The ranking logic should combine weather constraints with wardrobe tags. For example, a cotton shirt may score higher than a wool shirt in hot weather, while the same wool shirt could rise in score when the temperature drops. The app should explain these choices in simple language so the user understands why a piece was selected.

9.1 Question Flow For Outfit Selection

The user-facing questions should be short and practical: What is the occasion? What colors do you want to wear? What colors should be avoided? How formal should the outfit feel? Do you want something comfortable, sharp, or expressive? The answer set can be used to filter wardrobe items before scoring begins.

This is also where the app should let the user say no to colors they dislike. The color preference should not be a decorative setting; it should directly change the ranking so the selected wardrobe items match the user's taste.

9.2 How The Recommendation Engine Should Work

The engine should work in layers. First, filter out items that clearly do not fit the weather or the required dress code. Second, score color harmony and category compatibility. Third, apply user-specific preferences such as style persona, favorite colors, and fit. Finally, if an ML model exists, let it re-rank the best candidates rather than replace the whole rules system.

That order matters. If the model is asked to do everything from day one, it will be hard to debug. If the rules handle the obvious cases first, the model can focus on subtle personal preference and ranking quality.

10. No-Image-Output Rule

The final version must not generate an outfit image. The user should see the outfit as a set of real wardrobe items, their thumbnails, color chips, and an explanation card. This keeps the app honest: it recommends what the user actually owns instead of inventing a look that may not exist in the closet.

Because of that decision, any code that exists purely for image generation should be removed from the active path. In practice, that means the prompt generator in `frontend/app/lib/deepfashion2.js` should not be used, and the backend fields that store generated image URLs should be treated as legacy unless the team keeps them only for old records.

If a visual preview is still helpful, build it from the wardrobe item images that already exist in the database. That way the preview remains truthful, the UI stays useful, and the user can actually wear the outfit that is being suggested.

11. ML Model Training Plan

The ML layer should improve ranking and personalization, not replace the rules engine. A good first model learns which combinations a user is likely to save, wear, or rate highly. A great later model can also learn style taste, weather tolerance, and color preferences from user behavior.

The model should be trained offline, saved as an artifact, and loaded by the FastAPI backend for inference. The API should stay fast, so the training loop should not run on every request. Training is a background process; inference is the live request path.

11.1 What The Model Should Learn

Input type	Examples	What the model learns
Wardrobe metadata	Category, color, material, pattern, season tags, style tags, occasion tags.	Which item attributes are compatible with one another.
User profile	Style persona, body type, skin tone, preferred occasions.	How to personalize suggestions.
Weather context	Temperature, rain, humidity, wind, condition code.	When to prefer light, warm, or weather-safe items.
Behavior labels	Saved, liked, worn, skipped, rated.	What the user actually prefers in practice.
Combination features	Color harmony score, category fit, dress-code fit.	How to rank complete outfits.

11.2 Datasets That Can Be Used

The best approach is to combine public fashion datasets with your own app data. Public datasets help the model learn the general structure of clothing, while internal data teaches it the style preferences of real users. Since the team only needs a complete guide and not a research paper, it is enough to understand the role of each dataset instead of memorizing every class label.

Dataset	Why it is useful	Good for
DeepFashion2	Strong clothing category coverage and garment-level labels.	Category mapping, item detection, and taxonomy alignment.
Fashionpedia or similar attribute datasets	Useful when you need richer clothing attributes.	Pattern, material, sleeve, and style attributes.
Polyvore outfit compatibility data	Pairs and outfit sets are good for recommendation training.	Outfit compatibility and ranking.
FashionGen or captioned fashion data	Useful for style text, descriptions, and broad fashion context.	Text-to-style alignment and metadata enrichment.
Your own wardrobe logs	The most important source for personalization.	Save rate, wear history, and user preference learning.

If the team does not want to start with a complex neural model, that is fine. A strong baseline can be built with feature engineering plus a gradient-boosted ranking model. That baseline is often easier to

train, easier to explain, and easier to debug than a large vision model. The key is to learn ranking quality, not image generation.

11.3 Recommended Training Pipeline

Step A - Build the training table

Problem: Raw outfits and raw clothing items are not ready for training.

Solution: Flatten each outfit into a row of features: item category, color distances, weather context, occasion, and user feedback. If the dataset comes from a public source, normalize its labels into the Pincher taxonomy first.

Files: Recommended future ``backend/ml/prepare_dataset.py`` and ``backend/app/schemas/``.

Done when: You can export a clean CSV or parquet file ready for model training.

Step B - Train a baseline model

Problem: The app needs a ranking signal before the ML system becomes advanced.

Solution: Train a simple classifier or ranking model that predicts whether an outfit will be liked, saved, or worn. Start small and make the baseline easy to evaluate.

Files: Recommended future ``backend/ml/train.py``, ``backend/ml/evaluate.py``, and ``backend/ml/artifacts/``.

Done when: The model beats the rule-only baseline on a holdout set.

Step C - Expose inference through FastAPI

Problem: The frontend needs model scores without caring about model internals.

Solution: Load the saved model at API startup or lazily on first request, then call it from a service layer that ranks outfit candidates.

Files: Recommended future ``backend/app/services/recommender.py`` and ``backend/app/services/weather.py``.

Done when: The API returns ranked outfits with both rules-based and ML-based scores.

11.4 How To Judge Model Quality

Use top-k accuracy, NDCG, click or save rate, and rating lift as your main measures. In fashion recommendation, the top suggestion matters, but the second and third suggestions also matter because users like to browse alternatives. A model that produces helpful top-three results is often more valuable than a model that only wins on one rigid metric.

The simplest practical rule is this: if the ML model is confusing or hard to explain, it should only be allowed to re-rank results, not replace the whole recommendation pipeline. That keeps the product stable while the team keeps learning from real user data.

12. Implementation Roadmap From Start To Finish

This is the practical build order. If a teammate follows these steps in sequence, they will move from setup to a complete working version without skipping the foundations.

Phase	What to build	Result
1	Clean setup	Separate the active Next.js and FastAPI paths from legacy files and make the environment reproducible.
2	Database	Create and verify the PostgreSQL schema, indexes, and bootstrap script.
3	Authentication	Implement signup, login, profile lookup, token handling, and password hashing.
4	Wardrobe CRUD	Let users add, edit, filter, and delete real wardrobe items.
5	Frontend dashboard	Build the app shell, navigation, wardrobe views, and result screens.
6	Weather logic	Fetch weather and convert it into scoring rules for outfit selection.
7	No-image output	Remove generated-image dependency and show only wardrobe-based results.
8	ML ranking	Train, test, and serve a model that improves recommendation quality.
9	QA and launch	Add tests, documentation, and deployment checks.

Step 1 - Clean setup and repo hygiene

Problem: The repository currently contains both the new stack and legacy leftovers, which can confuse a new teammate.

Solution: Keep the active frontend and FastAPI backend as the real source of truth, and treat the deleted static frontend plus legacy Express/TypeScript backend as retired history.

Files: Frontend `app/`, backend `main.py`, and the legacy `backend/src/` and `backend/dist/` folders.

Done when: Everyone on the team can point to the same active code path without guessing which backend or frontend is real.

Step 2 - Database and persistence

Problem: The app must remember users, wardrobe items, and outfit history after the browser closes.

Solution: Use PostgreSQL tables for users, profiles, wardrobe items, and outfit collections. Add indexes on the columns you filter most often.

Files: Backend `src/schema.sql`, `init\_db.py`, and `db.py`.

Done when: Signup, wardrobe creation, and outfit saving work against real tables.

Step 3 - Authentication and identity

Problem: Users need a secure way to register and return later without losing their data.

Solution: Hash passwords with bcrypt, issue JWT tokens, and protect private endpoints with bearer-token checks.

Files: Backend `auth.py`, `main.py`, and the frontend login/signup pages.

Done when: A logged-in user can fetch `/auth/me` and see their own account.

Step 4 - Wardrobe ingestion

Problem: Recommendations are useless if the closet data is incomplete or messy.

Solution: Let the user upload item photos and metadata, then store category, color, season, style, and occasion tags.

Files: Frontend `app/dashboard/upload/page.jsx`, `app/lib/imageUtils.js`, and backend wardrobe routes in `main.py`.

Done when: Users can create, update, filter, and delete wardrobe items.

Step 5 - Frontend dashboard and result screens

Problem: The user needs a clean interface to browse items and inspect recommendations.

Solution: Use the dashboard layout as the shell and connect the route pages for wardrobe, outfit builder, results, history, saved looks, insights, analytics, profile, and settings.

Files: Frontend `app/dashboard/layout.jsx` and the route files under `frontend/app/dashboard/`.

Done when: The dashboard feels like a real product, not a pile of disconnected pages.

Step 6 - Weather-aware scoring

Problem: The app should not recommend the same outfit in every weather condition.

Solution: Fetch weather, apply clothing constraints, and rank pieces by temperature, rain, wind, humidity, occasion, and color harmony.

Files: New weather service code, `colorHarmony.js`, and the recommendation endpoint in the backend.

Done when: The top outfit changes sensibly when the weather changes.

Step 7 - Remove image generation from the live flow

Problem: A fake generated outfit image would drift away from the user's real wardrobe.

Solution: Keep wardrobe thumbnails and item cards, but remove the image-generation prompt path from the active recommendation flow.

Files: `frontend/app/lib/deepfashion2.js`, backend outfit fields, and any UI buttons or services that call image generation.

Done when: The app only shows real wardrobe items and no generated outfit image is needed to understand the result.

Step 8 - ML training and inference

Problem: Rules are a good start, but they do not fully capture personal taste.

Solution: Train a ranking model on public fashion datasets plus your own wardrobe and behavior logs, then expose inference through FastAPI.

Files: Recommended future `backend/ml/` and `backend/app/services/` folders.

Done when: The API can re-rank outfit options using learned preference signals.

Step 9 - Testing, deployment, and handoff

Problem: A teammate needs confidence that the app works outside the original developer's machine.

Solution: Add tests, document env vars, confirm the startup order, and prepare deployment instructions for both frontend and backend.

Files: Frontend and backend READMEs, tests, and any CI or deployment config you add.

Done when: A new teammate can clone the repo, set env vars, run the app, and follow the guide without help.

13. Testing, Deployment, And Handoff

The last mile matters. A system that is hard to start or impossible to verify is not actually complete, even if the features are technically present.

13.1 What To Test

Test signup, login, token expiration behavior, wardrobe CRUD, outfit saving, weather fetching, color scoring, and the final recommendation flow. Also test empty states, because a brand-new wardrobe should not crash the app. In the frontend, verify the mobile layout, sidebar navigation, form validation, and dashboard page transitions.

13.2 What To Document For Deployment

Document the backend host and port, the database connection string, the JWT secret, the weather API key if one is used, and the frontend API base URL. If the team deploys the frontend and backend separately, write down the exact CORS and environment settings so there is no mystery later.

The backend README should describe the real FastAPI startup command instead of the old placeholder instructions, and the frontend README should be updated from the create-next-app starter text to a Pincher-specific guide.

13.3 Handoff Checklist

Checkpoint	Pass condition
Authentication	A user can sign up, log in, and fetch their profile.
Wardrobe data	Items can be created, edited, filtered, and deleted.
Weather logic	The recommendation changes when the weather changes.
Wardrobe-only output	No generated image is required or shown.
Color logic	The app explains why the selected colors work together.
ML readiness	A model artifact can score or re-rank outfits.
Docs	A teammate can follow the guide without asking for a separate explanation.

14. Glossary

Wardrobe item	A single clothing piece saved by the user, such as a shirt, trouser, jacket, or shoe.
----------------------	---

Outfit collection	A saved set of wardrobe items that form one recommended look.
Color harmony	How well the colors in an outfit work together.
Style persona	A simplified style profile such as minimalist, classic, or streetwear.
Occasion tag	A label that describes where the outfit is suitable, such as office or party.
Weather context	The temperature, wind, rain, and humidity information used to adapt the recommendation.
Ranking model	A model that orders outfit options from best to least suitable.